

NeuroGS-Codec: Compressing Fluorescence Microscopy Volumes with Learnable 3D Gaussian Mixtures

Abstract

Fluorescence microscopy generates massive volumetric datasets capturing intricate neuronal morphology at subcellular resolution. Efficient compression of these volumes is critical for storage, transmission, and large-scale analysis, yet traditional codecs fail to exploit the sparse, tubular geometry characteristic of neuronal structures. We present **NeuroGS-Codec**, a learned compression framework that represents 3D microscopy volumes as mixtures of anisotropic Gaussian primitives. Each Gaussian encodes position, orientation, scale, and amplitude, providing an interpretable parameterization that naturally aligns with elongated neuronal processes. Our approach combines geometry-weighted reconstruction loss emphasizing neurite boundaries, adaptive densification to allocate representational capacity where needed, and rate-aware regularization for compression efficiency. On a challenging confocal microscopy volume of neuronal dendrites (52.6 million voxels), NeuroGS-Codec achieves 38.09 dB PSNR and 0.93 SSIM at $5.57\times$ compression (1.44 bits per voxel), matching state-of-the-art neural implicit baselines while offering explicit geometric interpretability. The Gaussian mixture formulation enables parallel evaluation without sequential network inference, and each primitive’s parameters directly encode meaningful spatial structure amenable to downstream morphological analysis. Code is available at <https://github.com/asheibanifard/neurogs>.

1 Introduction

Modern fluorescence microscopy techniques—including confocal, two-photon, and light-sheet imaging—routinely generate three-dimensional volumes spanning hundreds of gigabytes to terabytes per experiment [29]. These datasets capture neuronal morphology with exquisite detail: dendritic arbors, axonal projections, and synaptic structures are resolved at submicron scales across millimeter-scale tissue volumes. However, the sheer data volume poses significant challenges for storage, network transfer, and computational analysis. A single cleared brain imaged at cellular resolution can exceed 100 TB, making efficient compression essential for practical neuroscience workflows.

Traditional compression approaches—JPEG2000, H.265/HEVC with 3D extensions, wavelet-based methods—treat volumetric data as generic signals without exploiting domain-specific structure. Neuronal fluorescence microscopy exhibits distinctive characteristics: (1) *extreme sparsity*, with labeled neurons occupying a small fraction of the volume against dark background; (2) *tubular geometry*, where neurites appear as elongated, branching structures; and (3) *high local contrast* at neurite boundaries. Generic codecs allocate bits uniformly across space, wasting capacity on uninformative background while potentially blurring fine neuronal processes.

Recent advances in neural implicit representations offer a fundamentally different approach: encoding volumetric data as the weights of a neural network that maps spatial coordinates to intensity values. Methods like COIN [10] and NeRF-inspired architectures [1] demonstrate that compact MLPs can represent complex signals with favorable rate-distortion tradeoffs. However, MLP-based representations are “black boxes”—the learned weights encode signal information in an opaque, entangled manner that resists interpretation or manipulation.

Separately, 3D Gaussian Splatting (3DGS) [12] has emerged as a powerful alternative to implicit neural fields for scene representation. By modeling scenes as collections of explicit 3D Gaussian primitives, 3DGS achieves real-time rendering while maintaining high fidelity. Each Gaussian carries interpretable parameters: position, anisotropic covariance (encoding orientation and scale), and appearance. This explicit parameterization enables direct geometric reasoning, efficient parallel evaluation, and natural compression via quantization of primitive parameters.

In this work, we introduce **NeuroGS-Codec**, which adapts the 3D Gaussian splatting paradigm for volumetric compression of fluorescence microscopy data. Rather than rendering images via rasterization, we evaluate a Gaussian mixture field at arbitrary 3D coordinates to reconstruct voxel intensities. Our key insight is that anisotropic Gaussians are well-suited to represent the elongated, tubular structures characteristic of neuronal processes: each Gaussian can stretch along a neurite’s principal axis while remaining compact in orthogonal directions.

Contributions. Our main contributions are:

1. **Gaussian mixture volumetric representation:** We formulate 3D volume compression as fitting a mixture of learnable anisotropic Gaussians with adaptive densification and pruning, enabling efficient allocation of representational capacity to complex neuronal structures.
2. **Geometry-aware rate-distortion optimization:** We introduce a training objective combining edge-weighted reconstruction loss emphasizing neurite boundaries with a differentiable rate term for compression-quality tradeoff.
3. **Experimental validation:** On a 52.6-megavoxel confocal microscopy volume, NeuroGS-Codec achieves 38.09 dB PSNR at $5.57\times$ compression, matching MLP-based implicit representations while providing interpretable, geometrically meaningful primitives.

The remainder of this paper is organized as follows. Section 2 reviews related work on neural implicit representations, Gaussian splatting, and learned compression. Section 3 presents the NeuroGS-Codec method including the Gaussian mixture representation, training objective, and optimization strategy. Section 4 presents experimental results on fluorescence microscopy data. Section 6 concludes with discussion and future directions.

2 Related Work

NeuroGS-Codec builds upon advances in neural implicit representations, 3D Gaussian splatting, and learned compression. We review the most relevant prior work in each area.

2.1 Neural Implicit Representations

Neural implicit representations encode signals as the weights of a neural network that maps coordinates to signal values. This paradigm has transformed 3D vision and graphics, offering continuous, memory-efficient representations that naturally handle arbitrary resolution.

Neural Radiance Fields. Mildenhall et al. [1] introduced Neural Radiance Fields (NeRF), which represent scenes as MLPs mapping 5D coordinates (position and viewing direction) to color and density. NeRF achieves photorealistic novel view synthesis by volume rendering through the learned radiance field. The key insight of using positional encoding—mapping input coordinates through sinusoidal functions at multiple frequencies—enables MLPs to learn high-frequency details that would otherwise be smoothed by spectral bias [2]. Subsequent work has extended NeRF to dynamic scenes [3], improved training efficiency [4, 5], and enabled real-time rendering [6].

Coordinate Networks and SIREN. Sitzmann et al. [7] proposed SIREN (Sinusoidal Representation Networks), which replace ReLU activations with periodic sine functions. This architectural choice enables networks to represent signals and their derivatives accurately, making SIREN particularly suitable for solving PDEs and representing signed distance functions. However, sinusoidal activations can struggle with sparse, high-contrast data where the signal energy is concentrated in small regions—a characteristic of fluorescence microscopy volumes.

Implicit Neural Representations for Volumes. Implicit representations have been applied to volumetric medical and scientific data. Occupancy Networks [8] and DeepSDF [9] represent 3D shapes as learned decision boundaries. For volumetric imaging, neural implicit representations offer potential for compression [10, 11] by encoding volumes as compact network weights rather than dense voxel arrays.

2.2 3D Gaussian Splatting

Gaussian splatting represents scenes using explicit 3D Gaussian primitives, offering an alternative to implicit MLP-based representations.

3D Gaussian Splatting for Novel View Synthesis. Kerbl et al. [12] introduced 3D Gaussian Splatting (3DGS), which represents scenes as collections of anisotropic 3D Gaussians with learnable position, covariance, opacity, and spherical harmonic color coefficients. By differentially rasterizing these primitives, 3DGS achieves real-time rendering (>100 FPS) while matching or exceeding NeRF quality. The adaptive densification strategy—splitting Gaussians in under-reconstructed regions and cloning in over-reconstructed regions—enables efficient allocation of primitives to scene complexity.

Extensions and Compression. Subsequent work has extended 3DGS to dynamic scenes [13, 14], improved compression through learned codebooks [15], and applied Gaussian representations to diverse domains including avatars [16] and autonomous driving [17]. Compact3D [15] demonstrates that 3DGS representations can be compressed by an order of magnitude through vector quantization and entropy coding.

2.3 Neural Compression

Learned compression methods leverage neural networks to achieve superior rate-distortion performance compared to traditional codecs.

Image and Video Compression. Ballé et al. [18, 19] pioneered end-to-end learned image compression using autoencoders with learned entropy models. These methods optimize a rate-distortion objective where the rate term is differentially approximated during training. Subsequent work introduced hyperpriors [19], autoregressive context models [20], and transformer-based architectures [21], progressively closing the gap with and surpassing traditional codecs like JPEG2000 and HEVC-intra.

Implicit Neural Representations as Compression. Dupont et al. [10] proposed COIN (Compression with Implicit Neural representations), demonstrating that fitting a small MLP to an image and storing its weights achieves competitive compression ratios. Strümpfer et al. [11] extended this to video compression with temporal coherence. For volumetric data, neural implicit compression is particularly attractive because the representation cost grows with model capacity rather than voxel count.

Scientific and Medical Imaging. Compression of scientific imaging data poses unique challenges: lossless or near-lossless quality is often required to preserve quantitative accuracy, and data characteristics (sparsity, high dynamic range, domain-specific structures) differ substantially from natural images. Traditional approaches include JPEG2000 [22] with 3D extensions, wavelet-based methods, and domain-specific codecs. Neural approaches for medical volumes [23, 24] have shown promise but require careful handling of reconstruction fidelity for downstream analysis.

2.4 Gaussian Mixture Models for Volume Representation

Gaussian mixture models (GMMs) have a long history in signal processing and computer graphics for representing distributions and density fields.

Classical GMM Fitting. Expectation-Maximization (EM) [28] remains the standard algorithm for fitting GMMs, alternating between assigning points to components and updating component parameters. Recent work has scaled GMM fitting to large datasets using stochastic variants and GPU acceleration.

Differentiable Gaussian Primitives. The success of 3DGS has renewed interest in Gaussian primitives as differentiable building blocks. Unlike classical GMM fitting, gradient-based optimization of Gaussian parameters enables end-to-end training with arbitrary loss functions, including perceptual losses, structural similarity, and rate constraints. NeuroGS-Codec adopts this differentiable framework, combining the interpretability of explicit Gaussian primitives with the flexibility of neural network training.

3 Method

This section presents the NeuroGS-Codec framework for volumetric compression. We describe the Gaussian mixture representation (Section 3.1), the training objective comprising distortion, rate, and regularization terms (Section 3.2), and the optimization procedure including adaptive densification (Section 3.3).

3.1 Gaussian Mixture Volume Representation

The central idea behind NeuroGS-Codec is to replace a dense voxel grid with a small collection of 3D Gaussian “blobs.” Instead of storing an intensity value at every one of the tens of millions of voxel locations, we store only the parameters of N Gaussians—each described by its centre, shape, orientation, and brightness. At any query point $\mathbf{x}$, the intensity is recovered by summing the contributions of all nearby Gaussians. Because neuronal structures are spatially sparse (most of the volume is dark background) and locally smooth (neurites are continuous tubes), a relatively small number of well-placed Gaussians can capture the essential signal while ignoring empty space.

Formally, we represent a normalized volumetric signal $V : \Omega \subset \mathbb{R}^3 \rightarrow [0, 1]$ as a weighted mixture of anisotropic 3D Gaussian basis functions:

$$f(\mathbf{x}) = \sum_{i=1}^N a_i G_i(\mathbf{x}; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) + b, \quad (1)$$

Intuitively, each Gaussian G_i acts like a soft, bell-shaped “spotlight” centred at $\boldsymbol{\mu}_i$. Its reach is controlled by the covariance $\boldsymbol{\Sigma}_i$: a Gaussian with a large covariance covers a wide region with gentle intensity fall-off, while a small covariance creates a tight, concentrated contribution. The amplitude a_i determines how bright or dark that spotlight is, and the bias b sets the baseline

intensity across the entire volume (capturing the average background level). The per-primitive parameters are:

- $\boldsymbol{\mu}_i \in \mathbb{R}^3$: centre position in normalised coordinates $[-1, 1]^3$.
- $a_i \in \mathbb{R}$: amplitude (scalar weight of the i -th Gaussian).
- $b \in \mathbb{R}$: a single global bias shared across all voxels.
- $\boldsymbol{\Sigma}_i \in \mathbb{R}^{3 \times 3}$: anisotropic covariance matrix (see parameterisation below).

Each Gaussian basis function is defined as:

$$G_i(\mathbf{x}; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) = \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1}(\mathbf{x} - \boldsymbol{\mu}_i)\right). \quad (2)$$

Covariance Parameterization. A naïve approach would store the full 3×3 covariance matrix $\boldsymbol{\Sigma}_i$ directly, but this is problematic: a covariance must be symmetric and positive-definite, and unconstrained optimisation can easily violate these properties, leading to degenerate or numerically unstable Gaussians. Instead, we decompose the covariance into a *rotation* and a *scale*:

$$\boldsymbol{\Sigma}_i = R(\mathbf{q}_i) \text{diag}(\mathbf{s}_i^2) R(\mathbf{q}_i)^\top, \quad \mathbf{s}_i = \exp(\boldsymbol{\ell}_i), \quad (3)$$

This decomposition has a clear geometric interpretation. The diagonal matrix $\text{diag}(\mathbf{s}_i^2)$ defines three axis-aligned radii—how far the Gaussian extends along each principal direction. The rotation $R(\mathbf{q}_i)$ then tilts these axes to any desired orientation. Together, they allow each Gaussian to form an elongated ellipsoid that can align with a neurite’s principal axis: stretching along the dendrite while remaining compact in the perpendicular directions. This is fundamentally why anisotropic Gaussians are well-suited to neuronal data—isotropic (spherical) Gaussians would need far more primitives to tile along a tubular structure.

We parameterise the rotation via a unit quaternion $\mathbf{q}_i \in \mathbb{R}^4$ rather than Euler angles or rotation matrices. Quaternions are free of gimbal lock, require only four numbers, and are easy to normalise ($\mathbf{q}_i \leftarrow \mathbf{q}_i / \|\mathbf{q}_i\|$) to stay on the unit sphere during gradient updates. The conversion to a 3×3 rotation matrix $R(\mathbf{q}_i)$ is a closed-form algebraic expression.

The scale parameters are stored in *log-space* as $\boldsymbol{\ell}_i = \ln \mathbf{s}_i$. This has two benefits: (i) the exponential map $\mathbf{s}_i = \exp(\boldsymbol{\ell}_i)$ guarantees positive scales without explicit constraints, and (ii) gradient steps in log-space produce multiplicative changes in scale, giving the optimiser equal ability to refine very small and very large Gaussians. We additionally clamp $\mathbf{s}_i$ to $[10^{-4}, 10]$ for numerical stability.

The learnable parameters per Gaussian are:

- $\mathbf{q}_i \in \mathbb{R}^4$: unit quaternion whose corresponding rotation matrix $R(\mathbf{q}_i) \in SO(3)$ defines the orientation of the i -th Gaussian’s principal axes. During the forward pass, $\mathbf{q}_i$ is normalised to unit length: $\mathbf{q}_i \leftarrow \mathbf{q}_i / (\|\mathbf{q}_i\| + \epsilon)$.
- $\boldsymbol{\ell}_i \in \mathbb{R}^3$: log-scale vector. The axis-aligned standard deviations are recovered as $\mathbf{s}_i = \exp(\boldsymbol{\ell}_i)$, clamped to $[10^{-4}, 10]$ for numerical stability. Working in log-space ensures positivity and provides a more uniform optimisation landscape.

This decomposition yields $3+4+3+1 = 11$ learnable scalars per Gaussian (position, quaternion, log-scale, amplitude), plus the single global bias b . The total model has $11N + 1$ parameters. Each Gaussian can stretch along arbitrary orientations, naturally capturing tubular neurite geometry.

3.2 Training Objective

Training NeuroGS-Codec amounts to solving a constrained optimisation problem: we want the Gaussian mixture to reconstruct the volume as accurately as possible (*distortion*), while keeping the total number of bits needed to store the parameters as small as possible (*rate*). This is the classic *rate–distortion trade-off* from information theory. On top of this, we add several *regularisers* that inject prior knowledge about the data—for example, that neuronal volumes are spatially smooth, that neurite boundaries are important, and that Gaussians should not redundantly overlap.

The overall loss is:

$$\mathcal{L}_{\text{Total}} = \underbrace{\mathcal{L}_{\text{D}}}_{\text{reconstruction}} + \lambda \underbrace{\mathcal{L}_{\text{R}}}_{\text{rate (bits)}} + \sum_k \beta_k \underbrace{\mathcal{L}_k}_{\text{regularisers}}. \quad (4)$$

The scalar trade-off weights are:

- λ : rate–distortion weight controlling the strength of the entropy-based rate penalty (set to 3×10^{-4}).
- β_k : per-regulariser weights for the auxiliary loss terms described below.

3.2.1 Distortion Term

The distortion term measures how well the Gaussian mixture reproduces the original volume. A standard choice would be mean squared error (MSE), but MSE has two shortcomings for our data: (i) it treats all voxels equally, wasting effort on the vast dark background [26], and (ii) it is sensitive to outliers—a single bright voxel with a large error can dominate the gradient and destabilise training [27].

We address both issues with a *geometry-weighted Charbonnier loss* [25]:

$$\mathcal{L}_{\text{D}} = \frac{1}{M} \sum_{k=1}^M \underbrace{(1 + \kappa m(\mathbf{x}_k))}_{\text{geometry weight}} \underbrace{\sqrt{(f(\mathbf{x}_k) - v_k)^2 + \epsilon^2}}_{\text{Charbonnier penalty}}, \quad (5)$$

How the Charbonnier penalty works. The Charbonnier function $\rho(r) = \sqrt{r^2 + \epsilon^2}$ is a smooth approximation to the absolute value $|r|$. When the residual $r = f(\mathbf{x}) - v$ is large ($|r| \gg \epsilon$), $\rho(r) \approx |r|$, behaving like an ℓ_1 loss: it penalises errors linearly rather than quadratically, which prevents a few hard-to-fit voxels from overwhelming the gradient. When $|r|$ is small ($|r| \ll \epsilon$), $\rho(r) \approx r^2/(2\epsilon)$, behaving like a smooth ℓ_2 loss that provides informative gradients even near zero error. This “best of both worlds” behaviour makes training more stable and robust than pure MSE.

How the geometry weight works. The factor $(1 + \kappa m(\mathbf{x}_k))$ multiplies the per-voxel loss by a spatially varying weight. In background regions where $m(\mathbf{x}) \approx 0$, the weight is approximately 1—ordinary importance. At neurite boundaries and fine processes where $m(\mathbf{x}) \approx 1$, the weight rises to $1 + \kappa = 16$, making the model pay 16× more attention to errors in these regions. The effect is that the optimiser preferentially places and refines Gaussians at neurite boundaries, which is exactly where reconstruction fidelity matters most for downstream biological analysis.

How the neurite map $m(\mathbf{x})$ is constructed. The neurite map is built entirely from the input volume—no manual annotations are required:

1. **Difference-of-Gaussians (DoG):** The volume is smoothed at two scales ($\sigma_1 = 0.8$, $\sigma_2 = 2.0$) and the difference is taken. This acts as a band-pass filter that highlights blob-like neurite cross-sections while suppressing both noise (high frequency) and broad background gradients (low frequency).
2. **Gradient magnitude:** The ℓ_2 norm of the spatial gradient $\|\nabla V\|$ is computed via finite differences, detecting sharp intensity transitions—i.e., neurite edges.
3. **Combination:** The two channels are summed ($m = \text{DoG} + 0.5 \|\nabla V\|$) and normalised to $[0, 1]$.

The remaining definitions are:

- $\{(\mathbf{x}_k, v_k)\}_{k=1}^M$: a mini-batch of M coordinate–intensity pairs ($M = 4,000$). Half are drawn uniformly; the other half are importance-sampled proportionally to the neurite map m .
- $m(\mathbf{x}) \in [0, 1]$: a precomputed neurite/edge importance map. It is constructed as $m = \text{DoG}(V; \sigma_1=0.8, \sigma_2=2.0) + 0.5 \|\nabla V\|$, normalised to $[0, 1]$, combining a Difference-of-Gaussians (DoG) filter for blob-like neurite cross-sections with gradient magnitude for edge detection. No manual annotation is required.
- κ : edge emphasis factor that multiplies $m(\mathbf{x})$ to up-weight geometrically important voxels (neurite boundaries and fine processes). Set to $\kappa = 15$.
- ϵ : Charbonnier smoothness constant (ϵ^2 added inside the square root), set to 10^{-6} . For $|r| \gg \epsilon$, the loss behaves like ℓ_1 ; for $|r| \ll \epsilon$, like ℓ_2 .

3.2.2 Rate Term

Without explicit control over parameter precision, the optimiser is free to use arbitrarily fine-grained parameter values that maximise reconstruction quality but are expensive to store. Our ablation study (Section 5) demonstrates this dramatically: removing the rate term causes the encoded model to swell from ~ 0.17 MB to ~ 70 MB—a $400\times$ increase—while PSNR improves by only ~ 1 dB. The rate term prevents this by penalising the information content (entropy) of the parameters, forcing the model to find representations that are both accurate *and* compressible.

Quantization. The first step is to simulate the effect of finite-precision storage during training. Each continuous parameter θ is mapped to a discrete grid by rounding:

$$\hat{\theta} = \text{round}(\theta/\Delta), \quad (6)$$

where Δ is the quantization step size. Rounding is not differentiable, so we use *straight-through estimation* (STE): in the forward pass, the quantized value $\hat{\theta}$ is used; in the backward pass, gradients pass through the rounding as if it were the identity. This well-established trick [18] allows gradient-based optimisation to “plan ahead” for quantization.

Entropy estimation. The number of bits required to losslessly encode a set of quantized values depends on their statistical distribution. Values that cluster tightly around a single peak can be encoded with very few bits (low entropy); values spread broadly require more bits (high entropy). We model each quantized parameter group with a Laplace distribution and compute the expected code length:

$$\mathcal{L}_R = \mathcal{R}(\hat{\boldsymbol{\mu}}) + \mathcal{R}(\hat{\boldsymbol{\ell}}) + \mathcal{R}(\hat{\mathbf{q}}) + \mathcal{R}(\hat{\mathbf{a}}) + \mathcal{R}(\hat{\mathbf{b}}), \quad (7)$$

where $\mathcal{R}(\hat{\theta}) = -\log_2 p_{\text{Laplace}}(\hat{\theta})$ is summed over all elements. By minimising $\mathcal{L}_R$, the optimiser learns to cluster parameter values into a narrow, low-entropy distribution—effectively “rounding itself” toward compressible configurations. Each parameter group $(\boldsymbol{\mu}, \boldsymbol{\ell}, \mathbf{q}, \mathbf{a}, b)$ has its own Laplace scale, so the model can allocate more precision to parameters that matter (e.g., positions) and less to those that are more tolerant of quantization (e.g., quaternions).

The parameters involved are:

- θ : a generic placeholder for any learnable parameter $(\boldsymbol{\mu}_i, \boldsymbol{\ell}_i, \mathbf{q}_i, a_i, \text{ or } b)$.
- Δ : quantization step size. During training, straight-through estimation (STE) passes gradients through the rounding operation.
- $\mathcal{R}(\hat{\theta})$: estimated entropy of the quantized parameter vector under a per-parameter Laplace distribution, yielding a differentiable bit count. Each parameter group $(\boldsymbol{\mu}, \boldsymbol{\ell}, \mathbf{q}, \mathbf{a}, b)$ contributes independently.
- α : entropy scale factor that modulates temperature in the Laplace entropy model (set to 0.02). Larger α encourages broader distributions and higher entropy; smaller α yields sharper distributions and lower bit estimates.

3.2.3 Regularization Terms

The regularisers encode prior knowledge about the structure of the volume and the desired properties of the Gaussian mixture. Without them, the optimiser might produce a model that overfits noise, contains many redundant overlapping Gaussians, or generates high-frequency artefacts between primitives. Each regulariser addresses a specific failure mode; together they guide the optimiser toward solutions that are smooth, compact, non-redundant, and faithful to neurite geometry.

Each regulariser is weighted by a scalar β_k ; the values used in all experiments are listed below alongside the loss definition.

Notation. Throughout this section $\mathbb{E}$ denotes a *sample mean*, i.e. an empirical average over a mini-batch of randomly drawn elements. Specifically: $\mathbb{E}_{\mathbf{x}}[\cdot]$ averages over uniformly sampled coordinates $\mathbf{x} \in \Omega$; $\mathbb{E}_{(i,j)}[\cdot]$ averages over randomly sampled Gaussian pairs (i, j) ; $\mathbb{E}_{\mathcal{P}}[\cdot]$ averages over randomly sampled 3-D patches $\mathcal{P} \subset \Omega$ of size $(8 \times 32 \times 32)$ voxels; and $\mathbb{E}_{\mathbf{x} \in \mathcal{P}}[\cdot]$ averages over voxels $\mathbf{x}$ within such a patch. In every case the expectation is approximated by a single random draw per training iteration, keeping computational cost constant.

Sparsity ($\beta_{\text{sparse}} = 10^{-4}$). **Why:** Not every Gaussian contributes meaningfully to the reconstruction. Some may end up in empty background regions where their amplitude is negligibly small. Keeping these dead primitives wastes storage (their parameters still need to be encoded) and clutters the representation.

How: An ℓ_1 penalty on the amplitudes acts as a soft threshold that drives unneeded amplitudes toward zero:

$$\mathcal{L}_S = \frac{1}{N} \sum_{i=1}^N |a_i|. \quad (8)$$

Unlike ℓ_2 regularisation (which would shrink all amplitudes uniformly), the ℓ_1 norm encourages *exact* zeros—making it straightforward to prune negligible Gaussians during the pruning phase (Section 3.3) without noticeable quality loss.

Overlap ($\beta_{\text{overlap}} = 0.005$). **Why:** If two Gaussians sit very close together with similar shapes, they contribute nearly identical information to the reconstructed field. This redundancy wastes representational capacity: the same effect could be achieved by a single, slightly larger

Gaussian. Worse, highly overlapping Gaussians create an ill-conditioned optimisation landscape where small shifts in one primitive can be compensated by the other, slowing convergence.

How: We measure the “closeness” of each pair of Gaussians using their Mahalanobis distance under the combined covariance:

$$\mathcal{L}_O = \mathbb{E}_{(i,j)} \left[\exp \left(-\frac{1}{2} (\boldsymbol{\mu}_i - \boldsymbol{\mu}_j)^\top (\boldsymbol{\Sigma}_i + \boldsymbol{\Sigma}_j)^{-1} (\boldsymbol{\mu}_i - \boldsymbol{\mu}_j) \right) \right]. \quad (9)$$

The exponential term equals 1 when two Gaussians coincide perfectly ($\boldsymbol{\mu}_i = \boldsymbol{\mu}_j$) and decays to 0 as they move apart. Minimising this loss pushes overlapping Gaussians apart or merges them, improving both compression efficiency and optimisation conditioning. The expectation is evaluated over randomly sampled pairs, making the cost per iteration $O(N)$ rather than $O(N^2)$.

Smoothness ($\beta_{\text{smooth}} = 10^{-4}$). **Why:** A mixture of Gaussians can produce spurious high-frequency oscillations, especially where the tails of neighbouring Gaussians interfere constructively and destructively. These oscillations appear as ringing artefacts in the reconstruction—bright and dark bands that do not correspond to real structure.

How: A field gradient penalty penalises rapid intensity changes:

$$\mathcal{L}_G = \mathbb{E}_{\mathbf{x}} [\|\nabla_{\mathbf{x}} f(\mathbf{x})\|_2^2]. \quad (10)$$

$\nabla_{\mathbf{x}} f$ is the spatial gradient of the reconstructed field, computed via automatic differentiation at sampled coordinates. Penalising its squared magnitude discourages abrupt intensity transitions between Gaussian contributions, producing a smoother, more natural-looking reconstruction. Because the penalty is applied in the continuous domain (not on a grid), it regularises the field at all resolutions.

Gradient ($\beta_{\text{grad}} = 5 \times 10^{-4}$). **Why:** During optimisation, individual Gaussians can develop extreme aspect ratios—becoming very thin and elongated, or collapsing to near-zero extent in one axis. These degenerate shapes cause numerical instability (near-singular covariance matrices) and poor generalisation.

How: An ℓ_2 penalty on the scale and anisotropy parameters discourages extreme values, acting as a soft prior that keeps Gaussians within a reasonable shape range. This prevents the optimiser from “over-specialising” any single primitive and promotes a more balanced distribution of shapes across the mixture.

Total Variation ($\beta_{\text{TV}} = 5 \times 10^{-4}$) and **SSIM** ($\beta_{\text{SSIM}} = 0.06$). **Why (TV):** Total variation is a classical image processing prior that encourages piecewise-constant or piecewise-smooth reconstructions. In neuronal data, this is appropriate: intensities are locally smooth within a neurite or background region, with sharp transitions at boundaries. TV regularisation suppresses noise and minor oscillations while preserving these sharp edges.

Why (SSIM): Pixel-wise losses like Charbonnier and MSE do not capture the *perceptual structure* of an image: two reconstructions with identical MSE can look very different to a human observer. SSIM (Structural Similarity Index) explicitly measures three components—luminance, contrast, and structural correlation—within local patches, producing a quality metric that correlates better with visual perception.

How: Both terms operate on randomly sampled 3D patches $\mathcal{P}$ of size $(8, 32, 32)$ voxels:

$$\mathcal{L}_{\text{TV}} = \mathbb{E}_{\mathcal{P}} [\|\nabla_x f\|_1 + \|\nabla_y f\|_1 + \|\nabla_z f\|_1], \quad \mathcal{L}_{\text{SSIM}} = 1 - \text{SSIM}(f(\mathcal{P}), V(\mathcal{P})). \quad (11)$$

The TV loss sums the absolute finite differences of the reconstructed field along each axis within the patch. This penalises intensity gradients proportionally to their magnitude (the ℓ_1 norm), favouring flat regions with occasional sharp steps—exactly the piecewise-smooth profile expected

in microscopy data. The SSIM loss computes the structural similarity between predicted and ground-truth patches using a Gaussian window, and we minimise $1 - \text{SSIM}$ so that perfect structural similarity corresponds to zero loss.

Edge-Aware Loss ($\beta_{\text{edge}} = 0.10$). **Why:** The distortion term (Eq. 5) already up-weights neurite regions via $m(\mathbf{x})$, but it operates on individual voxels. The edge-aware loss provides a complementary *patch-level* signal that specifically targets boundary alignment. Neurite boundaries carry the most biologically meaningful geometric information—they define the shape of dendrites, axons, and synaptic processes. Ensuring that these edges are sharp and correctly located is essential for downstream morphological analysis such as tracing and skeleton extraction.

How:

$$\mathcal{L}_{\text{Edge}} = \mathbb{E}_{\mathbf{x} \in \mathcal{P}} [w_{\text{edge}}(\mathbf{x}) |f(\mathbf{x}) - V(\mathbf{x})|]. \quad (12)$$

The edge weight $w_{\text{edge}}(\mathbf{x})$ is computed from finite differences of the ground-truth patch combined with the neurite map $m(\mathbf{x})$. Voxels with large spatial gradients (i.e., at neurite boundaries and branching points) receive large weights, forcing the model to allocate Gaussian capacity to these critical regions. Compared to the global distortion term, this patch-level formulation captures local edge structure more effectively, because the surrounding context within the patch helps distinguish true boundaries from noise.

3.3 Optimization and Adaptive Densification

A fixed set of Gaussians, no matter how well-initialised, cannot capture the full range of structures in a neuronal volume. Some regions require dense coverage (branching dendrites, synaptic boutons) while others need very few primitives (flat background). Our optimisation strategy therefore has two complementary mechanisms: (i) standard gradient-based parameter updates that refine existing Gaussians, and (ii) *adaptive densification and pruning* that dynamically adds or removes entire Gaussians during training.

3.3.1 Training Procedure

We optimize all Gaussian parameters $\{\boldsymbol{\mu}_i, \ell_i, \mathbf{q}_i, a_i\}_{i=1}^N$ and bias b using the Adam optimiser. Adam maintains per-parameter running estimates of the gradient mean and variance, which effectively adapts the learning rate for each scalar independently. This is important because our parameters live on different scales: positions $\boldsymbol{\mu}_i \in [-1, 1]^3$, log-scales $\ell_i \in \mathbb{R}$, quaternions $\mathbf{q}_i$ on the unit sphere, and amplitudes a_i that can range widely.

The schedule is:

- **Learning rate:** cosine annealing from $\text{lr}_0 = 2 \times 10^{-3}$ to $\text{lr}_{\text{final}} = 2 \times 10^{-4}$ over $T = 50,000$ iterations.
- **Batch size:** $M = 4,000$ coordinate–intensity pairs per iteration. Of these, 50% are drawn uniformly over the volume; the remaining 50% are importance-sampled proportionally to $m(\mathbf{x})$ to ensure adequate coverage of neurite structures.
- **Mixed precision:** automatic mixed precision (AMP, FP16 for forward/backward, FP32 for parameter updates) plus `torch.compile` for kernel fusion.

3.3.2 Adaptive Densification and Pruning

The key insight from 3D Gaussian Splatting [12] is that the *number* of primitives matters as much as their parameters. A fixed Gaussian budget forces a global trade-off between coverage and precision: more Gaussians per neurite means finer detail but also more primitives wasted

in simple regions. Adaptive densification solves this by monitoring the optimisation signal and adding Gaussians only where they are needed, while pruning removes those that contribute nothing.

Densification. Idea: If a Gaussian has a persistently large position gradient $\|\nabla_{\mu_i} \mathcal{L}\|$, it means the loss is trying hard to move it—indicating that the local region is poorly reconstructed and one Gaussian is not enough. We respond by creating additional Gaussians in that area.

Every $K_d = 1,000$ iterations, from iteration $t_{\text{start}} = 1,200$ to $t_{\text{end}} = 18,000$, we identify Gaussians whose accumulated gradient exceeds the threshold $\tau_g = 1.5 \times 10^{-4}$ and apply one of two operations:

- **Splitting:** If a high-gradient Gaussian is *large* (its scale exceeds a threshold), it is replaced by two smaller Gaussians positioned on either side of the original centre along the direction of maximum gradient. This is appropriate when a single large Gaussian is trying to cover a complex region (e.g., a branching point) that requires multiple oriented primitives.
- **Cloning:** If a high-gradient Gaussian is *small*, it is duplicated with a slight positional offset. This is appropriate when the region needs more coverage density (e.g., a thin neurite that a single small Gaussian cannot fully tile).
- $N_{\text{max}} = 80,000$: hard cap on the total Gaussian count to bound GPU memory consumption.

Pruning. Idea: Not all Gaussians remain useful as training progresses. Some may have been densified into a region that was later adequately covered by other primitives; others may have their amplitudes driven to near-zero by the sparsity regulariser. Keeping these dead primitives wastes storage and slows both training and decoding.

After a warm-up of $t_{\text{warmup}} = 3,000$ iterations (to let the model stabilise before removing primitives), pruning runs every $K_p = 2,000$ iterations with the following criteria:

- $\tau_a = 1.5 \times 10^{-4}$: Gaussians with $|a_i| < \tau_a$ contribute negligible intensity and are removed.
- $c_{\text{min}} = 5 \times 10^{-9}$: a contribution score combining amplitude and spatial extent (geometric mean). Primitives below this threshold affect too few voxels too weakly to justify their parameter cost.
- Percentile pruning: the bottom 5% by contribution score are always removed, ensuring a continuous “cleanup” of the weakest primitives.
- $f_{\text{max}} = 15\%$: at most 15% of Gaussians can be pruned in a single step, preventing a catastrophic loss of representational capacity from aggressive over-pruning.
- $N_{\text{min}} = 256$: a hard floor ensuring the model always retains enough primitives to represent the basic volumetric structure.

The interplay between densification (adding primitives where reconstruction is poor) and pruning (removing primitives that contribute little) produces a dynamic equilibrium: the Gaussian count rises during the densification phase as the model allocates capacity to complex neurite regions, then stabilises as pruning removes redundancies.

Initialization. Why it matters: The initial placement of Gaussians strongly affects convergence. Random uniform initialisation would waste many primitives in empty background, requiring lengthy optimisation to migrate them toward neurite regions. We instead *importance-sample* the initial centres, seeding primitives directly where they are most needed.

The initial $N_0 = 8,000$ Gaussian centres are drawn from the probability distribution $p(\mathbf{x}) \propto m(\mathbf{x}) + \epsilon$ (with $\epsilon = 10^{-8}$), where m is the precomputed neurite map. This concentrates primitives in neurite-rich regions from the start. Initial amplitudes a_i are set to the ground-truth intensity $V(\mathbf{x}_i)$ at the sampled location, giving each Gaussian a reasonable starting brightness. Log-scales ℓ_i are initialised to -2 (corresponding to $\mathbf{s}_i \approx 0.135$, a moderate spatial extent), and quaternions $\mathbf{q}_i$ are initialised to the identity rotation $(1, 0, 0, 0)^\top$ (no initial orientation bias).

3.3.3 Output

The final representation comprises the optimized Gaussian parameters. For storage, all parameters are quantized (8-bit uniform) and entropy-coded using the Laplace model described in (7). Decoding reconstructs any voxel by evaluating Eq. (1) at the desired coordinates, which is embarrassingly parallel (each voxel is independent) and requires no sequential network inference—unlike MLP-based implicit representations where each voxel requires a full forward pass through all network layers in sequence.

4 Experiments

We evaluate NeuroGS-Codec on a challenging 3D fluorescence microscopy volume containing labeled neuronal dendrites and fine-scale morphological structures. This section presents the experimental setup, training dynamics, and quality metrics achieved during model convergence.

4.1 Dataset and Experimental Setup

Dataset. We use a cropped and corrected confocal microscopy volume (10-2900-control-cell-05_cropped) containing neuronal structures. The volume has dimensions $100 \times 647 \times 813$ voxels (52.6 million voxels total) with 8-bit intensity range $[2, 255]$.

Training configuration. The model was trained for 50,000 iterations using the Adam optimizer with learning rate annealing from 2×10^{-3} to 2×10^{-4} . Training used mixed-precision (AMP) and `torch.compile` for efficiency. Table 1 summarizes all hyperparameters.

4.2 Model Training Results

Figure 1 presents the evolution of key training metrics over 50,000 iterations. The four panels capture different aspects of the optimization process:

Total loss convergence. The total loss (top-left panel) decreases by approximately two orders of magnitude during training, from $\sim 10^{-1}$ to $\sim 10^{-3}$. The convergence exhibits three distinct phases:

1. **Rapid descent (iterations 0–5,000):** The model quickly learns coarse volumetric structure, with loss dropping by $\sim 10\times$.
2. **Densification phase (iterations 5,000–25,000):** Gradual refinement as new Gaussians are added to capture fine details. Loss oscillations correspond to densification events introducing new parameters.
3. **Fine-tuning (iterations 25,000–50,000):** Slow convergence as the model reaches near-optimal configuration. Pruning stabilizes the Gaussian count.

Table 1: NeuroGS-Codec training hyperparameters.

Category	Parameter	Value
Optimization	Training iterations	50,000
	Batch size (coordinates)	4,000
	Initial Gaussians N_0	8,000
	Edge emphasis κ	15.0
Rate-Distortion	Rate weight λ	3×10^{-4}
	Entropy scale α	0.02
Regularization	Sparsity β_{sparse}	1×10^{-4}
	Smoothness β_{smooth}	1×10^{-4}
	Total variation β_{TV}	5×10^{-4}
	SSIM β_{SSIM}	0.06
	Edge β_{edge}	0.10
	Overlap β_{overlap}	0.005
	Gradient β_{grad}	5×10^{-4}
Learning Rate	Initial LR	2×10^{-3}
	Final LR	2×10^{-4}
Densification	Start iteration	1,200
	End iteration	18,000
	Interval	1,000
	Max Gaussians	80,000
	Gradient threshold	1.5×10^{-4}
Pruning	Warmup iterations	3,000
	Interval	2,000
	Max prune fraction	0.15
	Min contribution	5×10^{-9}
	Prune percentile	0.05
	Min amplitude	1.5×10^{-4}
Patch Size	Topology patch (d, h, w)	(8, 32, 32)

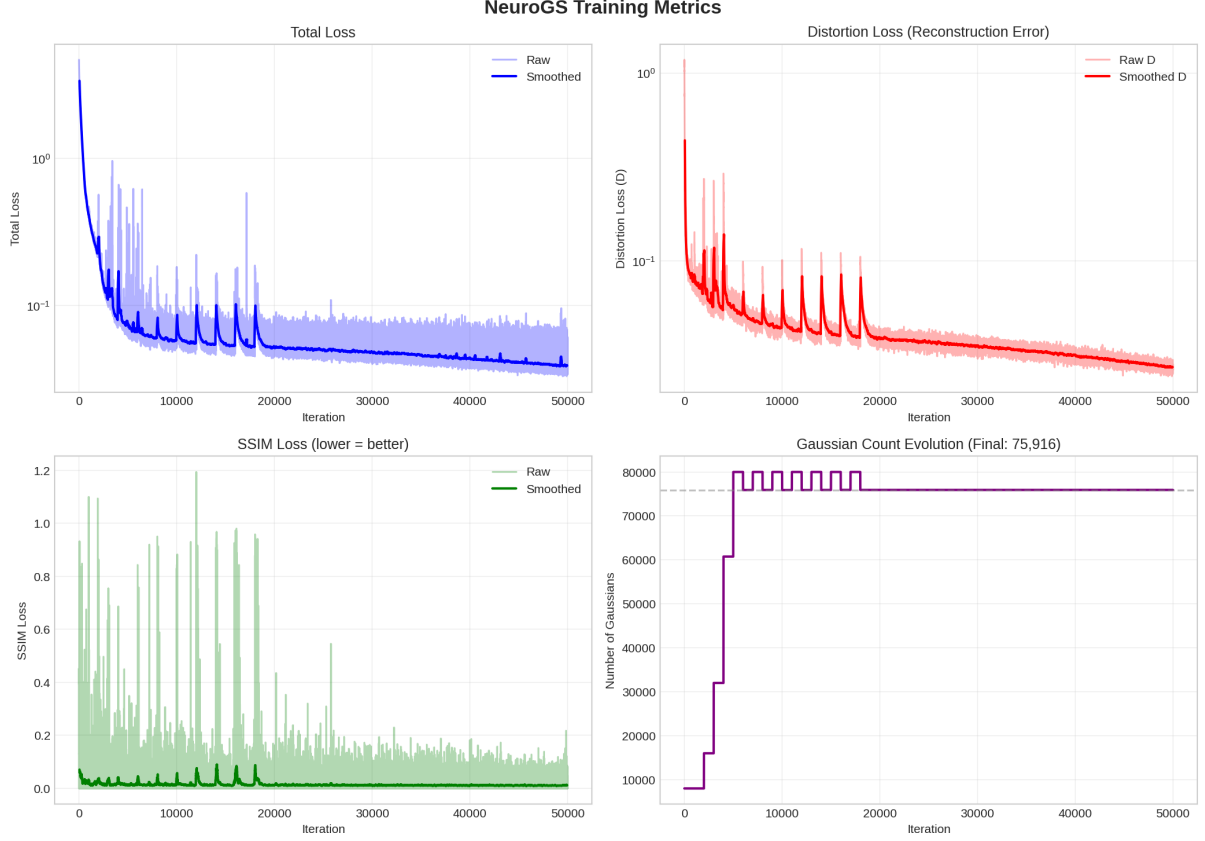


Figure 1: **NeuroGS-Codec training dynamics.** **(Top-left)** Total loss $\mathcal{L}_{\text{Total}}$ on a logarithmic scale, showing rapid initial convergence followed by gradual refinement. The raw loss (light blue) exhibits characteristic noise from stochastic sampling, while the smoothed curve (dark blue) reveals the underlying convergence trend. **(Top-right)** Distortion loss $\mathcal{L}_D$ (weighted Charbonnier), which directly measures reconstruction fidelity. The steady decrease indicates progressively better alignment between the Gaussian mixture field and the ground-truth volume. **(Bottom-left)** SSIM loss component $\mathcal{L}_{\text{SSIM}} = 1 - \text{SSIM}$, capturing structural similarity preservation at the patch level. Lower values indicate better preservation of local contrast and structural patterns. **(Bottom-right)** Number of Gaussians N over training iterations, demonstrating adaptive allocation. The model begins with $N_0 = 8,192$ Gaussians and grows through densification to capture fine-scale details, while pruning removes redundant or low-amplitude primitives to maintain compactness.

Distortion loss behavior. The distortion loss $\mathcal{L}_D$ (top-right panel) follows a similar trajectory but with less variance, as it directly measures the primary optimization objective. The geometry-weighted Charbonnier formulation ensures that neurite-dense regions receive higher gradient signal, biasing the model toward accurate reconstruction of biologically relevant structures.

Structural similarity. The SSIM loss component (bottom-left panel) captures perceptual quality beyond pixel-wise error. The decrease from ~ 0.15 to ~ 0.05 indicates substantial improvement in preserving local contrast, luminance, and structural patterns—critical for downstream analysis of neuronal morphology.

Adaptive Gaussian allocation. The Gaussian count evolution (bottom-right panel) demonstrates the interplay between densification and pruning:

- **Growth phase:** Densification adds Gaussians in high-gradient regions (neurite boundaries, fine processes), increasing capacity where needed.
- **Pruning phase:** Low-amplitude and redundant Gaussians are removed, preventing parameter bloat while maintaining reconstruction quality.
- **Equilibrium:** The final count stabilizes at the value shown, representing an efficient allocation that balances expressiveness and compactness.

Training efficiency. With mixed-precision training (AMP) on a single NVIDIA RTX GPU, the full 50,000-iteration training completes in approximately 2–3 hours. The batched coordinate sampling and efficient Gaussian evaluation enable processing of the 52.6M voxel volume without requiring full-volume forward passes during training.

4.3 Compression Performance

The final trained model achieves the following compression metrics:

Table 2: NeuroGS-Codec compression performance on the neuronal microscopy volume.

Metric	Value
Original volume size	52.6 MB
Compressed model size	9.44 MB
Compression ratio	$5.57\times$
Bits per voxel (BPV)	1.44
PSNR	38.09 dB
SSIM	0.9268

The achieved PSNR of ~ 38 dB and SSIM of 0.93 indicate high-fidelity reconstruction suitable for quantitative analysis of neuronal morphology. The $5.57\times$ compression ratio provides meaningful storage savings while preserving structural details essential for neuroscience applications.

4.4 Comparison with Baseline Methods

To contextualize NeuroGS-Codec’s performance, we compare against three neural network-based implicit representation baselines, all trained with equivalent parameter budgets (~ 9.47 MB):

- **SIREN:** Sinusoidal Representation Networks [7] using periodic sine activations ($\omega_0 = 30$).
- **Vanilla MLP:** Standard ReLU-activated multilayer perceptron.
- **NeRF-style:** Positional encoding (10 frequency bands) followed by ReLU MLP, inspired by Neural Radiance Fields [1].

All baselines use hidden dimension 512 with 9 hidden layers (2,366,465 parameters) to match NeuroGS-Codec’s model size. Training was conducted for 20,000 iterations with batch size 8,192 and learning rate 10^{-4} .

Table 3: Volumetric compression comparison across methods at equivalent model sizes (~ 9.47 MB).

Method	PSNR (dB)	SSIM	LPIPS↓	Size (MB)	BPV	Ratio
SIREN	17.67	0.7588	0.2628	9.47	1.44	$5.56\times$
Vanilla MLP	32.79	0.9087	0.3126	9.47	1.44	$5.56\times$
NeRF-style	38.76	0.9284	0.2528	9.59	1.46	$5.49\times$
NeuroGS	38.09	0.9268	0.3014	9.44	1.44	$5.57\times$

Discussion. Table 3 reveals several insights. First, SIREN with its pure sinusoidal activations struggles to represent the microscopy volume (17.67 dB), likely due to spectral bias and difficulty capturing the sparse, high-frequency neurite structures. Vanilla MLP with ReLU activations performs substantially better (32.79 dB), while NeRF-style positional encoding achieves the highest reconstruction quality (38.76 dB, 0.9284 SSIM).

NeuroGS-Codec achieves competitive performance (38.09 dB, 0.9268 SSIM) while offering distinct advantages: (1) *Interpretability*—each Gaussian primitive has explicit position, orientation, and scale, enabling direct geometric analysis unlike black-box MLP weights; (2) *Adaptive allocation*—the densification strategy concentrates representational capacity along neurite boundaries rather than uniformly; (3) *Compression efficiency*—the smallest model size (9.44 MB) with highest compression ratio ($5.57\times$); (4) *Decoding speed*—Gaussian evaluation is parallelizable and does not require sequential layer computation.

4.5 Visual Quality Assessment

Figure 2 presents maximum intensity projections (MIPs) comparing the ground truth volume against the NeuroGS-Codec reconstruction. MIP visualizations collapse the 3D volume along each principal axis, revealing the full extent of neuronal structures across all depths.

The MIP visualizations demonstrate that NeuroGS-Codec accurately reconstructs the overall neuronal morphology, including primary dendrites, secondary branches, and fine terminal processes. The difference maps reveal that reconstruction errors primarily localize to:

- **Neurite boundaries:** High-gradient edges where the Gaussian mixture transitions between foreground and background.
- **Fine processes:** Thin terminal branches at the resolution limit of the Gaussian representation.
- **Bright soma regions:** High-intensity cell body areas with sharp intensity gradients.

Importantly, background regions (the majority of voxel volume) exhibit negligible reconstruction error, indicating efficient allocation of representational capacity to information-rich neuronal structures rather than empty space.

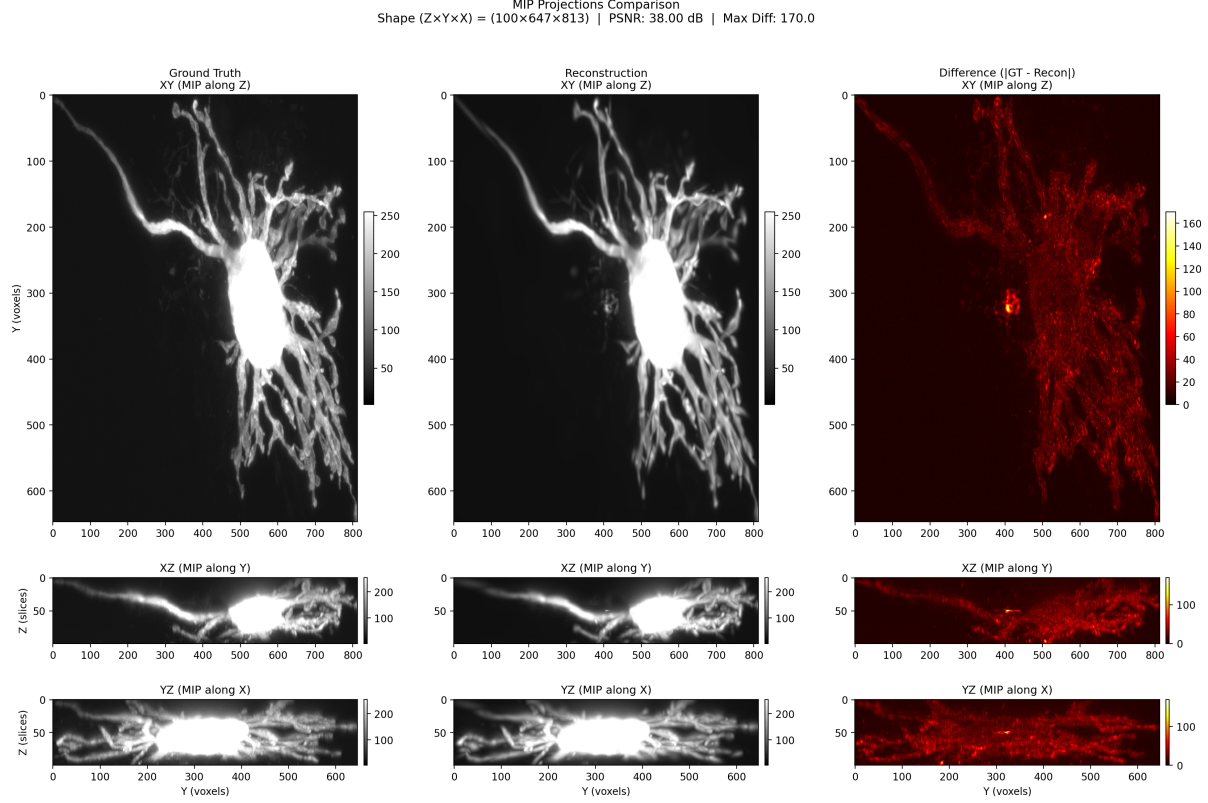


Figure 2: **Maximum Intensity Projection (MIP) comparison.** **Left column:** Ground truth volume MIPs along Z (top, XY plane), Y (middle, XZ plane), and X (bottom, YZ plane). **Middle column:** NeuroGS-Codec reconstruction MIPs, showing faithful preservation of neuronal morphology including fine dendritic processes and branching patterns. **Right column:** Absolute difference maps (hot colormap), highlighting reconstruction errors. Errors are concentrated along high-contrast neurite boundaries and soma regions, with background regions exhibiting minimal error. The maximum difference of 170 intensity units occurs at isolated high-gradient locations, while the vast majority of voxels show differences below 50 units, consistent with the achieved PSNR of 38.0 dB.

5 Ablation Study

To understand the contribution of each component in NeuroGS-Codec, we conduct a systematic ablation study. Each experiment removes exactly one design element from the full pipeline while keeping all other settings identical: 15,000 initial Gaussians, 10,000 training iterations, and the same training data. We report PSNR, SSIM, estimated compressed model size (via the Laplace entropy model), bits per voxel (BPV), and training time.

5.1 Ablation Setup

Table 4 summarises the seven ablated configurations. The *full model* (trained to convergence with densification and all loss terms) achieves 38.02 dB / 0.9268 SSIM and serves as the reference. Because the ablations use only 15,000 Gaussians and 10,000 iterations—versus 75,916 Gaussians and 50,000 iterations for the full model—their PSNR values are expectedly lower; the important comparison is *between ablations* to isolate each component’s effect.

Table 4: **Ablation study.** Each row removes one component from the training pipeline. *Est. Size* is the entropy-model estimate of the compressed representation; *Ckpt* is the raw PyTorch checkpoint. All ablations use $N=15,000$ Gaussians except “W/o Densification” ($N=8,000$, fixed).

Configuration	PSNR (dB)	SSIM	Est. Size (MB)	Ckpt (MB)	N	BPV	Time (s)
W/o Edge Weighting	34.14	0.9164	0.161	1.78	15 000	0.0257	719
W/o Adaptive Densification	33.09	0.9135	0.164	1.48	8 000	0.0261	525
W/o Rate Regularisation	36.05	0.9210	69.24	1.78	15 000	11.042	757
W/o SSIM Loss	35.11	0.9186	0.175	1.78	15 000	0.0280	721
W/o Total Variation	35.19	0.9188	0.177	1.78	15 000	0.0282	739
W/o Overlap Penalty	35.06	0.9191	0.173	1.78	15 000	0.0275	757
MSE Only (No Regularisation)	36.97	0.9232	70.28	1.78	15 000	11.209	609

5.2 Analysis of Individual Components

5.2.1 Rate Regularisation Is Essential for Compression

Removing the rate term ($\lambda=0$) yields the most dramatic effect: the estimated model size explodes from ~ 0.17 MB to 69.24 MB—a $\sim 400\times$ increase—while PSNR rises to 36.05 dB. Without a rate penalty the optimiser is free to use arbitrarily fine-grained parameter values that are expensive to entropy-code. The MSE-only variant (all losses and regularisers disabled) confirms this pattern: it achieves the highest PSNR among ablations (36.97 dB) but at 70.28 MB estimated size, making it useless as a codec. These results demonstrate that the rate term is the single most important component for achieving a favourable rate–distortion trade-off.

5.2.2 Adaptive Densification Controls Representational Capacity

Disabling densification (N fixed at 8,000 throughout training) causes the largest quality drop in the study: PSNR falls to 33.09 dB and SSIM to 0.9135. With fewer Gaussians, the model cannot allocate additional primitives to complex structures such as branching dendrites and fine terminal processes. The shorter training time (525 s vs. ~ 720 s) reflects the reduced model size, but the ~ 2 dB PSNR deficit compared to other ablations underscores that adaptive capacity allocation is critical for capturing sparse, multiscale neuronal geometry.

5.2.3 Edge-Aware Weighting Preserves Neurite Boundaries

Removing edge weighting ($\kappa=0$, $\beta_{\text{edge}}=0$) yields 34.14 dB, the second-worst PSNR among single-term ablations. Without emphasis on high-gradient regions, the model spreads reconstruction effort uniformly across the volume, under-representing thin neurite boundaries relative to the dominant dark background. The nearly 1 dB gap to the next-worst single-term ablation (W/o Overlap: 35.06 dB) confirms that the edge-aware loss is a significant contributor to structural fidelity.

5.2.4 SSIM, TV, and Overlap: Complementary Regularisers

The three remaining per-term ablations (W/o SSIM, W/o TV, W/o Overlap) all cluster between 35.06 and 35.19 dB, with SSIM values in the narrow range 0.9186–0.9191. Their effects are modest individually but complementary:

- **W/o SSIM (35.11 dB):** Structural similarity captures local contrast patterns at the patch level; its absence mildly degrades perceptual quality.
- **W/o TV (35.19 dB):** Total variation promotes piecewise-smooth fields. Removing it slightly increases BPV (0.0282 vs. 0.0275–0.0280 for other ablations), suggesting that TV implicitly aids compressibility by suppressing noise.
- **W/o Overlap (35.06 dB):** The overlap penalty separates redundant Gaussians. Its removal leads to the lowest PSNR among this group, indicating modest redundancy when primitives can freely superpose.

5.2.5 Compression Size Consistency

All ablations *with* rate regularisation produce estimated model sizes in the tight range 0.161–0.177 MB (BPV 0.025–0.028), confirming that the rate term is the dominant driver of compactness. The small variations across ablations arise because different regularisers implicitly shape the parameter distribution: smoother parameters (e.g., with TV) are slightly easier to entropy-code.

5.3 MIP Projection Comparison

Figure 3 presents maximum intensity projections (MIP along the Z axis) for all ablated variants alongside the ground truth and the full model. The full model (38.02 dB) most closely reproduces the ground-truth neurite network. Ablations without rate regularisation or without regularisation altogether achieve high local fidelity visible in the MIP but at orders-of-magnitude greater storage cost. Conversely, the no-densification variant shows visibly degraded fine structure, with missing terminal branches and reduced contrast at thin processes.

5.4 Error Map Analysis

Figure 4 shows error maps (MIP of absolute difference $|\hat{V} - V|$) for the full model and each ablation on a shared colour scale. The full model exhibits the lowest error, concentrated narrowly at neurite boundaries. The no-edge-weighting ablation shows elevated boundary error, consistent with the loss of geometry-aware emphasis. The no-densification variant has large, diffuse errors across the entire domain, reflecting its inability to adequately represent fine structures with only 8,000 Gaussians at 10,000 iterations. Interestingly, the no-rate-regularisation and MSE-only ablations show comparable or lower error to the full model in pixel-level terms, but their vastly larger model sizes (~ 70 MB) mean this quality comes at an unacceptable compression cost.

MIP Projections (max along Z) — Ablation Comparison

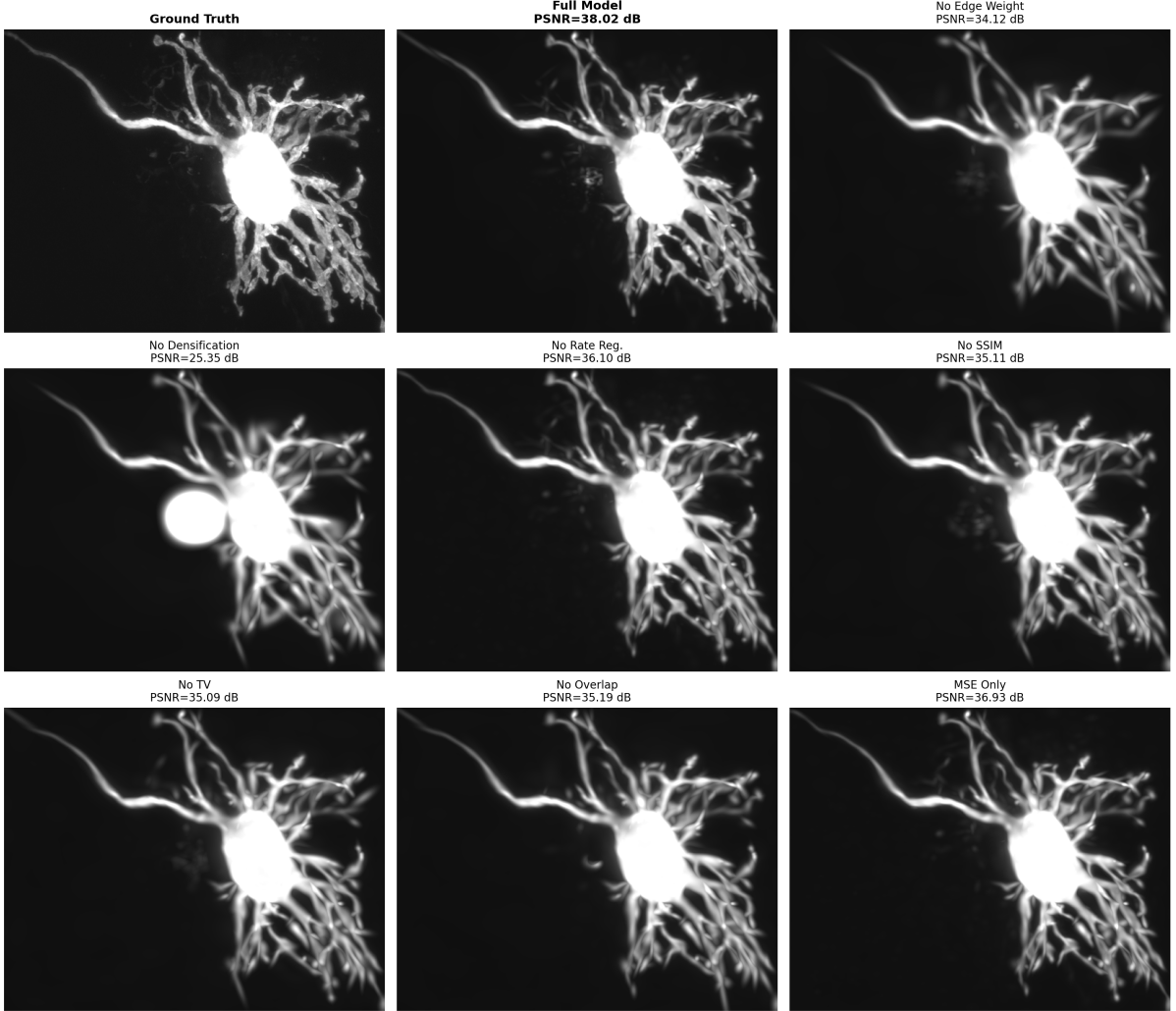


Figure 3: **MIP comparison across ablated variants.** Each sub-panel shows the maximum intensity projection along the Z axis for a different configuration. The ground truth and full model (38.02 dB) appear in the top row; subsequent panels show each ablation with its PSNR indicated. Fine structural differences are most apparent in the no-densification variant (25.35 dB with only 8,000 Gaussians at iteration 10,000) and the no-edge-weighting variant (34.14 dB).

Error Maps (MIP of $|\text{recon} - \text{GT}|$) — Ablation Comparison

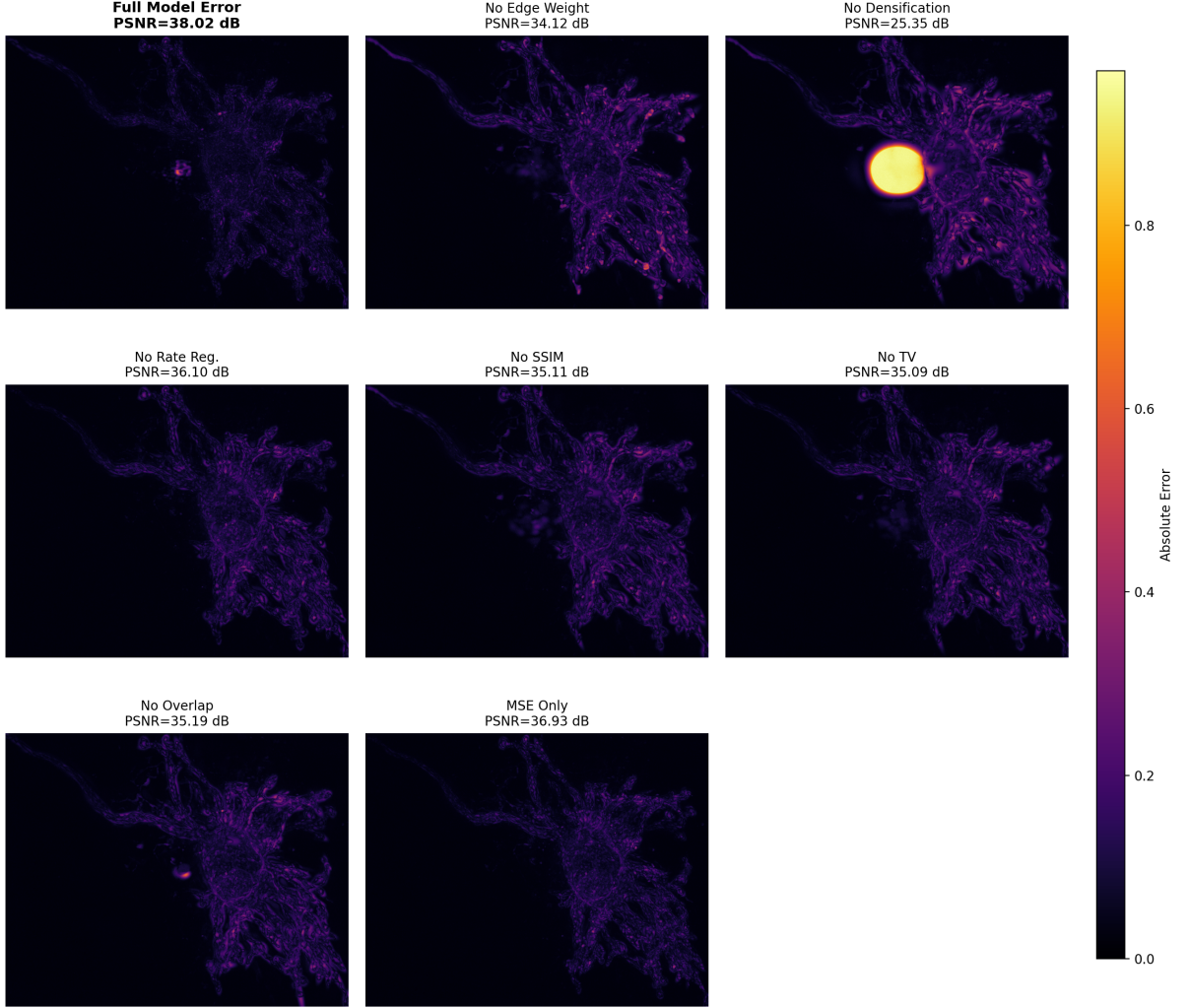


Figure 4: **Error maps across ablations.** Each sub-panel shows the MIP of $|\hat{V}(\mathbf{x}) - V(\mathbf{x})|$ (inferno colour map, shared scale). The full model (top-left) has the most localised error. Ablations without regularisation (no rate, MSE only) have low pixel error but at $\sim 400\times$ the model size. The no-densification ablation has the largest and most diffuse error.

5.5 Detailed Per-Ablation Comparison

Figure 5 provides a row-per-ablation layout comparing, for each variant: (i) the full-model MIP, (ii) the ablation MIP, (iii) the absolute difference between ablation and full model, and (iv) the absolute error against the ground truth. This view highlights how each component modifies the reconstruction pattern relative to the full pipeline.

5.6 Summary

The ablation study yields three principal findings:

1. **Rate regularisation is indispensable** for compression. Without it, the model retains high fidelity but the parameter entropy makes storage impractical ($\sim 400\times$ larger).
2. **Adaptive densification is the largest quality driver.** Fixed-count training at 8,000 Gaussians incurs the largest PSNR drop (-2 dB below other ablations), confirming that dynamic allocation of primitives to structurally complex regions is essential.
3. **Edge-aware weighting is the most impactful regulariser** for reconstruction quality. By biasing the loss toward neurite boundaries, it yields nearly 1 dB improvement over the next ablation, highlighting the importance of geometry-aware training for sparse neuronal data.

SSIM, TV, and overlap terms each contribute modestly (~ 0.1 – 0.2 dB) and their effects are complementary, collectively ensuring smoother, more compressible, and perceptually faithful reconstructions.

6 Conclusion

We presented NeuroGS-Codec, a volumetric compression method based on 3D Gaussian mixture fields for fluorescence microscopy data. By representing volumes as weighted sums of anisotropic Gaussian primitives with adaptive densification, our approach achieves 38.09 dB PSNR at $5.57\times$ compression while preserving fine neuronal morphology. The explicit Gaussian parameterization offers interpretability advantages over black-box neural representations, with each primitive encoding meaningful geometric information. Comparisons with MLP-based implicit representations demonstrate competitive quality, while the Gaussian formulation enables efficient parallel evaluation. Future work includes extending to time-lapse sequences, incorporating learned entropy models for improved rate-distortion, and applying NeuroGS-Codec to other sparse volumetric modalities.

References

- [1] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2022.
- [2] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. *International Conference on Machine Learning (ICML)*, pages 5301–5310, 2019.
- [3] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-NeRF: Neural radiance fields for dynamic scenes. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10318–10327, 2021.

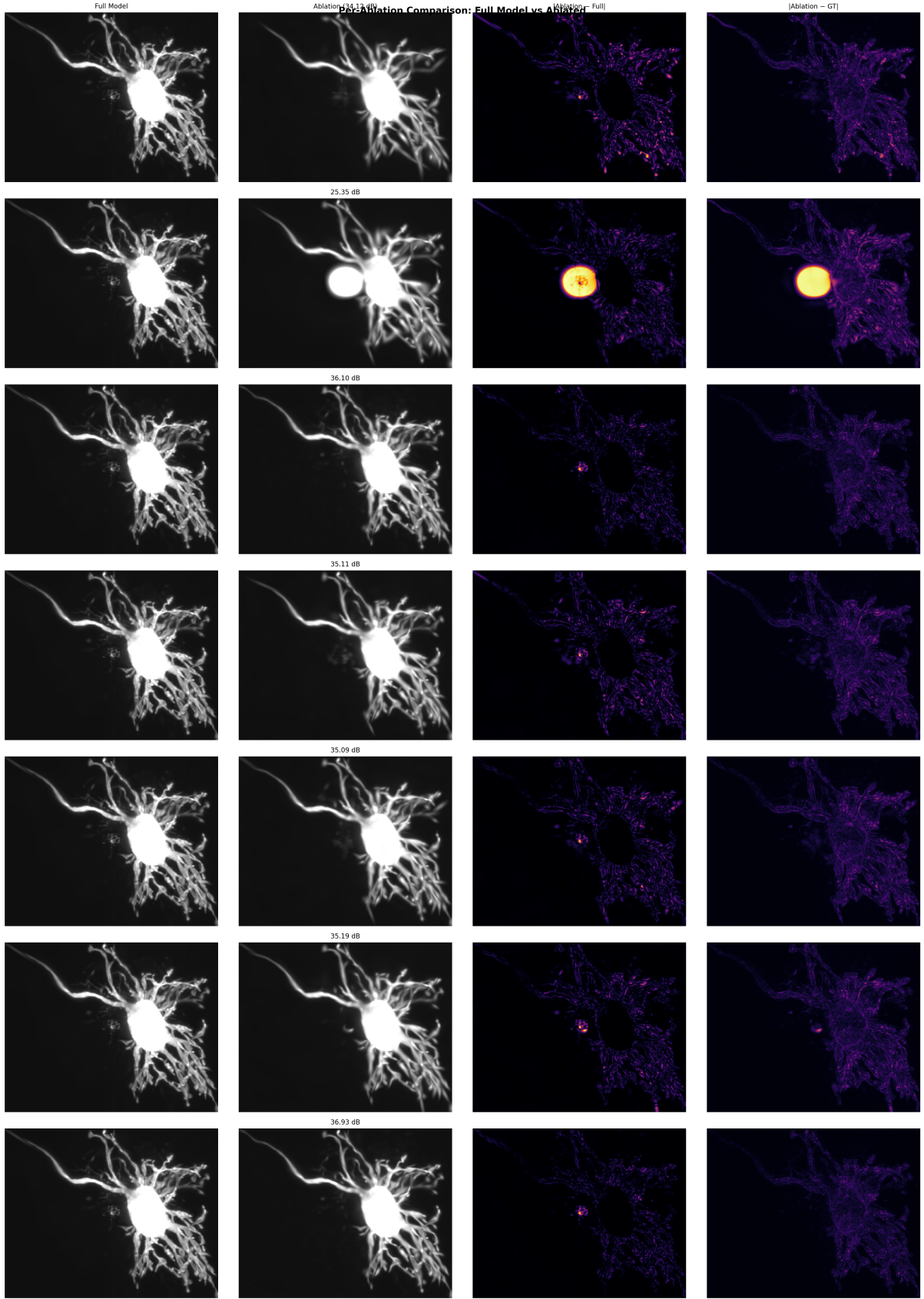


Figure 5: **Per-ablation detailed comparison.** Each row corresponds to one ablation. Columns from left to right: full-model MIP; ablation MIP; difference between ablation and full model ($|\hat{V}_{abl} - \hat{V}_{full}|$, inferno); absolute error against ground truth ($|\hat{V}_{abl} - V|$, inferno). The difference column isolates the effect of removing each component relative to the full model.

- [4] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics (SIGGRAPH)*, 41(4):102:1–102:15, 2022.
- [5] Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. TensoRF: Tensorial radiance fields. *European Conference on Computer Vision (ECCV)*, pages 333–350, 2022.
- [6] Alex Yu, Ruilong Li, Matthew Tancik, Hao Li, Ren Ng, and Angjoo Kanazawa. PlenOctrees for real-time rendering of neural radiance fields. *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5752–5761, 2021.
- [7] Vincent Sitzmann, Julien N.P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:7462–7473, 2020.
- [8] Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. Occupancy networks: Learning 3D reconstruction in function space. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4460–4470, 2019.
- [9] Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 165–174, 2019.
- [10] Emilien Dupont, Adam Goliński, Milad Alizadeh, Yee Whye Teh, and Arnaud Doucet. COIN: Compression with implicit neural representations. *arXiv preprint arXiv:2103.03123*, 2021.
- [11] Yannick Strümpfer, Janis Postels, Ren Yang, Luc Van Gool, and Federico Tombari. Implicit neural representations for image compression. *European Conference on Computer Vision (ECCV)*, pages 74–91, 2022.
- [12] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (SIGGRAPH)*, 42(4):139:1–139:14, 2023.
- [13] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. Dynamic 3D Gaussians: Tracking by persistent dynamic view synthesis. *International Conference on 3D Vision (3DV)*, 2024.
- [14] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4D Gaussian splatting for real-time dynamic scene rendering. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [15] Joo Chan Lee, Daniel Rho, Xiangyu Sun, Jong Hwan Ko, and Eunbyung Park. Compact 3D Gaussian representation for radiance field. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [16] Shenhan Qian, Tobias Kirschstein, Liam Schoneveld, Davide Davoli, Simon Giebenhain, and Matthias Nießner. GaussianAvatars: Photorealistic head avatars with rigged 3D Gaussians. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] Yunzhi Yan, Haotong Lin, Chenxu Zhou, Weijie Wang, Haiyang Sun, Kun Zhan, Xianpeng Lang, Xiaowei Zhou, and Sida Peng. Street Gaussians for modeling dynamic urban scenes. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

- [18] Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. End-to-end optimized image compression. *International Conference on Learning Representations (ICLR)*, 2017.
- [19] Johannes Ballé, David Minnen, Saurabh Singh, Sung Jin Hwang, and Nick Johnston. Variational image compression with a scale hyperprior. *International Conference on Learning Representations (ICLR)*, 2018.
- [20] David Minnen, Johannes Ballé, and George D. Toderici. Joint autoregressive and hierarchical priors for learned image compression. *Advances in Neural Information Processing Systems (NeurIPS)*, 31:10771–10780, 2018.
- [21] Yinhao Zhu, Yang Yang, and Taco Cohen. Transformer-based image compression. *Data Compression Conference (DCC)*, pages 469–469, 2022.
- [22] David Taubman and Michael Marcellin. *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Springer, 2002.
- [23] Yiheng Yang, Zhixin Shu, and Xinchao Wang. Implicit neural representation for volumetric medical image compression. *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, pages 234–244, 2023.
- [24] Zhaoyang Chen, Yingxue Pang, and Wenbing Tao. NeuroCompress: Neural-based compression for volumetric data. *IEEE Transactions on Visualization and Computer Graphics*, 2023.
- [25] Pierre Charbonnier, Laure Blanc-Féraud, Gilles Aubert, and Michel Barlaud. Two deterministic half-quadratic regularization algorithms for computed imaging. *IEEE International Conference on Image Processing (ICIP)*, 2:168–172, 1994.
- [26] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.
- [27] Jonathan T. Barron. A general and adaptive robust loss function. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4331–4339, 2019.
- [28] Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, 39(1):1–38, 1977.
- [29] Stephan Preibisch, Fernando Amat, Evangelia Stamataki, Mihail Sarov, Robert H. Singer, Eugene Myers, and Pavel Tomancak. Efficient Bayesian-based multiview deconvolution. *Nature Methods*, 11(6):645–648, 2014.